

SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

COURSE NAME: STATISTICS WITH R PROGRAMMING | COURSE CODE: ITA0404

COURSE OUTCOME COVERED: CO3 -- ASSESSMENT TOOL 2

CO3: Construct and manipulate data frames using reshaping, merging, and file I/O operations in R to prepare structured datasets for analysis (K3)

Assessment Tool Weightage: 30% | **Questions:** 5 | **Total Marks:** 50 | **Duration:** 1 Hour

Student Name: PAVITHRA.R

Reg. No: 192421369

Code of Conduct:

I PAVITHRA.R (192421369) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

Reg. No: 192421369

RUBRICS FOR CALCULATION

Criteria	Weightage	Q1 (10)	Q2 (10)	Q3 (10)	Q4 (10)	Q5 (10)	Total Marks (50)
Conceptual Understanding	30						
Accuracy of Answer	25						
Application of Knowledge	20						
Completeness of Response	15						
Clarity & Presentation	10						
Total per Question	10 Marks	/10	/10	/10	/10	/10	/ 50

1. Employee Project Allocation

TCS maintains two data files: `employee.csv` (Emp_ID, Name, Department) and `project.csv` (Emp_ID, Project, Hours_Worked).

Task:

- Read both CSV files into separate data frames.
- Merge the two data frames using Emp_ID.
- Identify employees who worked more than 160 hours.
- Add a new column Status with "High" if hours are above 160, otherwise "Normal".
- Write the updated data frame into `employee_project_report.csv`.

R Program Code Solution:

```
# 1. Read both CSV files into separate data frames
emp_df <- read.csv("employee.csv")
proj_df <- read.csv("project.csv")

# 2. Merge the two data frames using Emp_ID
merged_df <- merge(emp_df, proj_df, by = "Emp_ID")

# 3. Identify employees who worked more than 160 hours
high_hours_emp <- merged_df[merged_df$Hours_Worked > 160, ]
print("Employees working more than 160 hours:")
print(high_hours_emp)

# 4. Add new column Status ("High" if > 160, else "Normal")
merged_df$Status <- ifelse(merged_df$Hours_Worked > 160, "High", "Normal")

# 5. Write the updated data frame into CSV
write.csv(merged_df, "employee_project_report.csv", row.names = FALSE)
cat("Successfully exported report to employee_project_report.csv
")
```

2. Customer Transaction Analysis

Customer details are stored in `customers.csv` (Customer_ID, Name, City) and transaction details in `transactions.csv` (Customer_ID, Transaction_ID, Amount).

Task:

- Read both files.
- Merge the data frames using Customer_ID.
- Calculate the total transaction amount for each customer.
- Add a Category column: "Premium" for customers whose total exceeds ₹50,000, otherwise "Regular".
- Save the final report into `customer_transaction_report.csv`.

R Program Code Solution:

```
# 1. Read both files
cust_df <- read.csv("customers.csv")
trans_df <- read.csv("transactions.csv")

# 2. Merge data frames using Customer_ID
merged_df <- merge(cust_df, trans_df, by = "Customer_ID")

# 3. Calculate total transaction amount for each customer
total_amt <- aggregate(Amount ~ Customer_ID + Name + City, data = merged_df, FUN = sum)

# 4. Add Category column ("Premium" > 50000, else "Regular")
total_amt$Category <- ifelse(total_amt$Amount > 50000, "Premium", "Regular")

# 5. Save final report into CSV
write.csv(total_amt, "customer_transaction_report.csv", row.names = FALSE)
cat("Successfully exported report to customer_transaction_report.csv
")
```

3. IT Asset Management

Wipro maintains employee information (Emp_ID, Name, Department) and asset information (Emp_ID, Asset_ID, Asset_Type, Cost) in CSV files.

Task:

- Read employee and asset details from CSV files.
- Merge the data frames using Emp_ID.
- Find the total asset cost assigned to each department.
- Modify the Asset_Type value "Laptop" to "Company Laptop".
- Write the modified data into asset_management_report.csv.

R Program Code Solution:

```
# 1. Read employee and asset details
emp_df <- read.csv("employee.csv")
asset_df <- read.csv("asset.csv")

# 2. Merge data frames using Emp_ID
merged_df <- merge(emp_df, asset_df, by = "Emp_ID")

# 3. Find total asset cost assigned to each department
dept_cost <- aggregate(Cost ~ Department, data = merged_df, FUN = sum)
print("Total Asset Cost by Department:")
print(dept_cost)

# 4. Modify Asset_Type value 'Laptop' to 'Company Laptop'
merged_df$Asset_Type[merged_df$Asset_Type == "Laptop"] <- "Company Laptop"

# 5. Write modified data into CSV
write.csv(merged_df, "asset_management_report.csv", row.names = FALSE)
cat("Successfully exported report to asset_management_report.csv
")
```

4. Sales Performance Analysis

An organization stores sales information across two CSV files:

employee.csv: Emp_ID, Name, Team

sales.csv: Emp_ID, Month, Sales

Task:

- Read both files.
- Merge the data frames using Emp_ID.
- Identify employees whose sales exceed ₹50,000.
- Create a new column Performance containing "Excellent" for sales $\geq$ ₹60,000 and "Good" otherwise.
- Write the final data into sales_performance.csv.

R Program Code Solution:

```
# 1. Read both files
emp_df <- read.csv("employee.csv")
sales_df <- read.csv("sales.csv")

# 2. Merge data frames using Emp_ID
merged_df <- merge(emp_df, sales_df, by = "Emp_ID")

# 3. Identify employees whose sales exceed 50,000
high_sales <- merged_df[merged_df$Sales > 50000, ]
print("Employees with Sales > 50,000:")
print(high_sales)

# 4. Create Performance column ('Excellent' if >= 60000, else 'Good')
merged_df$Performance <- ifelse(merged_df$Sales >= 60000, "Excellent", "Good")

# 5. Write final data into CSV
write.csv(merged_df, "sales_performance.csv", row.names = FALSE)
cat("Successfully exported report to sales_performance.csv\n")
```

5. Project Attendance Analysis

A software company maintains employee details in `employee.csv` (`Emp_ID`, `Name`, `Project`) and attendance details in `attendance.csv` (`Emp_ID`, `Working_Days`, `Present_Days`).

Task:

- Read the two CSV files.
- Merge them using `Emp_ID`.
- Calculate the attendance percentage for every employee.
- Add a `Status` column: "Eligible" if attendance is at least 90%, otherwise "Review".
- Export the final employee attendance report to `attendance_report.csv`.

R Program Code Solution:

```
# 1. Read the two CSV files
emp_df <- read.csv("employee.csv")
att_df <- read.csv("attendance.csv")

# 2. Merge using Emp_ID
merged_df <- merge(emp_df, att_df, by = "Emp_ID")

# 3. Calculate attendance percentage
merged_df$Attendance_Percentage <- (merged_df$Present_Days / merged_df$Working_Days) * 100

# 4. Add Status column ('Eligible' if >= 90%, else 'Review')
merged_df$Status <- ifelse(merged_df$Attendance_Percentage >= 90, "Eligible", "Review")

# 5. Export final report to CSV
write.csv(merged_df, "attendance_report.csv", row.names = FALSE)
cat("Successfully exported report to attendance_report.csv\n")
```